Chapter 20 worked solutions • Influential operating systems

OS20-E01 • Spool bottleneck

1. CPUJ1 0–2, J2 2–4, J3 4–6;printerJ1 2–6, J2 6–10, J3 10–14. Output waits without interrupting the printer's current job.
2. Total14, CPU 6/14=3/7, printer 12/14=6/7. Serial3(2+4)=18. One printer still performs12 rather than 4 units of service.
3. Spooling retains jobs/data/queues to decouple producers/consumers. Multiprogramming schedules other ready CPU work while one waits. Spools alone supply neither multiple resident jobs nor scheduling/isolation. Full queues/timing substantiate utilization claims.

OS20-E02 • THE and Mach boundaries

1. Both decompose services/responsibilities. THE emphasizes abstraction/synchronization;Mach3 user servers additionally use protected address spaces/IPC. Modularity alone does not isolate faults.
2. The specified separate user-server IPC entails crossings. A THE layer call need not trap;Mach2.x retains much BSD code in kernel and differs.
3. EstablishC0;eachCi assumes lower contracts/stated interference, establishes itself, and preserves lower hidden state. Induction then applies. Cycles or direct uncontrolled lower-state changes violate premises. Layer, ring, process differ.

OS20-E03 • Atlas branches and faults

1. All-zero set uses branch 3, maxT5 selectsB. Other set has no t>T+1;branch 2 ignoresC'st0 and compares scoresA2, B5, selectingB. Omitting branch 3 leaves a legal state undefined.
2. 3597=7 • 512+13→2 • 512+13=1037 physical words, not bytes.
3. Both require valid mapping, correct contents, restartable faults, frame ownership. Modern multilevel tables, TLBs, COW, sharing, permissions, multicore synchronization add conditions. Atlas predictions do not guaranteeOPT minimum faults.

Chapter 20 worked solutions • Influential operating systems (continued)

OS20-E04 • XDS relocation/sharing

1. Eight virtual pages,32 frames.6161=3 • 2048+17→11 • 2048+17=22545 words.
2. Two shared+two • six private=14, not16/12. Code must be safely read-only/reentrant;mutable data remains separate. Multiple mappings do not privatize shared bytes.
3. No. This case relocates resident pages. Demand loading, restartable missing-page faults, and swapping policy are additional mechanisms. Separate units from residency policy.

OS20-E05 • Two-buffer conservation

1. After sends, free∅, queued{b0, b1}. After reply, free∅, queued{b1}, reply{b0}, total 2. If processing not yet answered, placeb0 only in processing instead.
2. No while reply owns it. After C1 consumes/returns, b0 becomes free and C3 can claim it. Each buffer has one state.
3. No. C1 awaitingS while S awaitsC1 forms circular wait. Primitive atomicity protects its transition, not whole transactions/system progress. Conservation and wait relationships need separate checks.

OS20-E06 • Exponential feedback

1. Allowed1/2/4/8, used1/2/4/4, remaining10/8/4/0:four dispatches. The final slice ends early on completion.
2. Feedback11+4 • .5=13;RR11+11 • .5=16.5. These are model costs, not historical speed measurements.
3. Full service throughk sums1+2+…+2^k=2^(k+1)−1. Blocking, promotion, competitors change burst/wait traces. Larger quanta alone do not prevent low-priority starvation. Do not mix different feedback rules.

Chapter 20 worked solutions • Influential operating systems (continued)

OS20-E07 • Multics segments/pages

1.1024=2^10, so18-bit offset splits8-bit page/10-bit displacement.2500=2 • 1024+452;PA7 • 1024+452=7620 words, after descriptor bound/rights checks. 2. Rings enforce privilege/entry;layers organize dependencies/abstractions. Modules can share a ring;six layers do not prove six domains. 3. Resolution identifies/references an object. Access lists, operation, execution context still authorize it. Shared writable segments still need synchronization. Mapping and authorization are separate steps.

OS20-E08 • MFT/MVT and base-limit

1. Two50 jobs consume64 each, wasting28 internally. Remaining128 cannot fit70 in one region. Variable holes total 120 but max50 cannot fit70, external fragmentation.
2. Validate0≤a<768.0→256,767→1023;768/−1 fail. Protected setup is essential;user-modifiable bounds destroy confinement.
3. CP supplies VM resource isolation;CMS runs as a single-user guest. Multiple guests do not make CMS one shared-memory multiuser kernel. Configuration, relocation, VM abstraction differ. Total free space does not replace a suitable interval.

OS20-E09 • Specified Mach3 copying

1. Request2 mode changes,1 address-space change,64 B copied. Reply2,1,8192 B. Total4,2,8256 B. Extra interrupts/scheduling are excluded;changed premises require recalculation.
2. Protection limits server directly corrupting client/kernel memory. Clients still handle timeout, failed state, reconnection, retry. Neither universal availability nor bug-free kernel follows.
3. Mach2.x retains much BSD in kernel;derivatives can reorganize services. Version and actual placement, not a Mach name, determine IPC costs. Mode, address-space, and byte counts differ.

Chapter 20 worked solutions • Influential operating systems (continued)

OS20-E10 • Names versus mechanisms

1. Cooperative yield/block-only switching may starveB forever. Preemptive progress requires serviceable timer, bounded kernel blocking, and scheduling that givesB time. Two windows prove neither processes nor protection.
2. No. Syntax assistance creates a recognizable request;trusted identity, target, right still determine access. Names can also change between checking/opening.
3. No. Languages improve expression/checks;small kernels can narrow trusted scope. Synchronization, ownership, authority, and failure contracts still need proof. Usability, isolation, progress, and overall correctness are distinct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 20 worked solutions • Influential operating systems (continued)

Verification code appendix: history_models.py

```
"""Self-contained historical mechanism models; not historical machine emulation."""
from collections import deque


def atlas_victim(pages):
    """Each row=(name,t,T), nonnegative sampled history. First row breaks ties."""
    if not pages or any(t<0 or T<0 for _,t,T in pages):raise ValueError('nonempty valid
history')
    if len({p[0] for p in pages})!=len(pages):raise ValueError('duplicate page')
    overdue=[p for p in pages if p[1]>p[2]+1]
    if overdue:return overdue[0][0],1
    inactive=[p for p in pages if p[1]!=0]
    if inactive:return max(inactive,key=lambda p:p[2]-p[1])[0],2
    return max(pages,key=lambda p:p[2])[0],3


def feedback(burst,initial=0):
    """q_i=2**i; no competitor, I/O, promotion or cost in timeline."""
    if burst<0 or initial<0:raise ValueError('negative work/level')
    remaining=burst;level=initial;total=0;rows=[]
    while remaining:
        q=2**level;used=min(q,remaining);remaining-=used;total+=used
        rows.append((level,q,used,total,remaining));level+=1
    return rows


def spool(jobs,cpu=2,printer=4):
    if jobs<0 or cpu<=0 or printer<=0:raise ValueError('invalid workload')
    cpuend=0;printend=0;rows=[]
    for job in range(1,jobs+1):
        start=cpuend;cpuend+=cpu;printstart=max(cpuend,printend);printend=printstart+printer
        rows.append((job,start,cpuend,printstart,printend))
    return rows


class Pool:
    """One buffer per request+reply exchange; deterministic atomic transitions."""
    def __init__(self,count=2):
        if count<0:raise ValueError('count')
        self.state={i:'free' for i in range(count)};self.queue=deque()
    def send(self):
        free=[i for i,s in self.state.items() if s=='free']
        if not free:raise BlockingIOError('pool exhausted')
        i=min(free);self.state[i]='queued';self.queue.append(i);return i
    def receive(self):
        if not self.queue:raise BlockingIOError('no request')
```

Chapter 20 worked solutions • Influential operating systems (continued)

```
        i=self.queue.popleft();self.state[i]='processing';return i
    def answer(self,i):
        if self.state.get(i)!='processing':raise ValueError('not server-owned')
        self.state[i]='reply'
    def reclaim(self,i):
        if self.state.get(i)!='reply':raise ValueError('no completed reply')
        self.state[i]='free'
    def valid(self):
        return len(set(self.queue))==len(self.queue) and set(self.queue)=={i for i,s in
self.state.items() if s=='queued'} and all(s in ['free','queued','processing','reply'] for s
in self.state.values())


def split_ge645(segment, word_offset):
    """Original GE645 two18-bit fields; segment-bound/protection checks separate."""
    if not 0<=segment<(1<<18) or not 0<=word_offset<(1<<18):
        raise ValueError('GE645 field outside18bits')
    return segment,word_offset//1024,word_offset%1024


def ge645_read(segment,word_offset,descriptors):
    """Paged-segment example, authority checks before demand-page resolution.
    Not a complete descriptor encoding/GE645 machine implementation.
    """
    number,page,within=split_ge645(segment,word_offset)
    desc=descriptors.get(number)
    if desc is None or page>=desc['bound_pages'] or not desc['read']:
        return 'protection-fault',None
    pte=desc['pages'].get(page)
    if pte is None or not pte['legal'] or not pte['read']:
        return 'protection-fault',None
    if not pte['present']:return 'page-fault',None
    return 'success',pte['frame']*1024+within


def protected_address(a,base=256,limit=768):
    if not 0<=a<limit:raise PermissionError('outside user segment')
    return base+a


if __name__=='__main__':
    for case in [[('A',6,3),('B',2,5),('C',0,9)],[('A',2,4),('B',1,6),('C',0,8)],[('A',0,2),
('B',0,5),('C',0,3)]]:print('Atlas',atlas_victim(case))
    print('feedback11',feedback(11));print('spool3',spool(3))
    p=Pool();a=p.send();b=p.send();i=p.receive();p.answer(i)
    print('before reply consume',p.state);p.reclaim(i);print('reclaimed',p.state,p.valid())
```

Chapter 20 worked solutions • Influential operating systems (continued)

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study